lab6 24.11.25

6a) WAP to implement Single list with following operations; Sort the linked list, Reverse the linked list, Concatenation of two linked list.

Pseudocode:

Sorting: (Bubble sort technique)

→ declare a variable tempdata to store and swap i, j values
→ use a for loop initialised by i=head, i→next=NULL, i=i→next
→ use another for loop to traverse across the linked list using the initialisation j=i→next ; j!=NULL ; j=j→next
→ check if i→data > j→data, execute the code

```
tempdata = i→data
i→data   = j→data
j→data   = tempdata
```

Reversing:

struct node *. reverselist (struct node *head) {

struct node *prev = NULL,

* curr = head. * next = NULL ;

→ use a while loop under the condition curr != NULL

→ set next = curr → next;

curr → next to prev, prev equals curr.

and curr equals next.

→ Given Linked list is reversed.

Concatenation:

→ check if head of 2nd linked list is null, then desired concatenated list is 1st linked list.

→ initialise a variable temp to head1

→ Run a while loop until temp → next != NULL by updating temp = temp → next

→ point temp → next to head 2

→ return the head1 which is providing with the concatenated linked list.

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
struct Node *head1 = NULL;
struct Node *head2 = NULL;
struct Node* createList(int n) {
    struct Node *head=NULL, *newNode, *temp;
    int data;
    if(n<=0) {
        printf("Number of nodes should be greater than 0\n");
        return NULL;
    }
```

```
for(int i=1 ; i<=n ; i++) {
    newNode = (struct Node*) malloc (sizeof(struct Node));
    if(newNode == NULL) {
        printf("Memory allocation failed \n");
        return head;
    }
    printf("Enter data for node %d ", i);
    scanf("%d", &data);
    newNode->data = data;
    newNode->next = NULL;
    IF (head == NULL).
        head = newNode;
    else
        temp->next = newNode;
    temp = newNode;
    }
    Printf("Linked list created successfully \n");
    return head;
}

Void displayList (struct Node *head){
    struct Node *temp = head;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    printf("Linked List : ");
    while (temp != NULL) {
        Printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```
void sortLinkedList (struct Node *head ) {
    struct Node *i, *j;
    int tempdata;
    if (head == NULL) {
        printf("List is empty, cannot sort.\n");
        return;
    }
    for (i=head; i->next != NULL; i=i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->data > j->data) {
                tempdata = i->data;
                i->data = j->data;
                j->data = tempdata;
            }
        }
    }
    printf("Linked list sorted successfully\n");
}

struct Node * reverseLinkedList( struct Node *head ) {
    struct Node *prev=NULL, *curr=head, *next= NULL;
    while (curr!=NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    printf("Linked list reversed successfully\n");
    return prev;
}

struct Node* concatenateLinkedList( struct Node *head1,
                                    struct Node *head2) {
    struct Node *temp;
    if (head1 == NULL) {
        return head2; }
```

```
    temp = head1;
    while (temp->next != NULL) {
        temp = temp->next; }
    temp->next = head2;
    printf(" linked lists concatenated successfully \n");
    return head1;
}

int main() {
    int choice, n;
    while(1) {
        printf(" 1. create list 1 \n");
        printf(" 2. create list2 \n");
        printf(" 3. sort list 1 \n");
        printf(" 4. reverse list 1 \n");
        printf(" 5. sort list 2 \n");
        printf(" 6. reverse list 2 \n");
        printf(" 7. concatenate \n");
        printf(" 8. Exit \n");
        printf(" Enter choice : ");
        scanf("%d", &choice);
        switch(choice) {
        case 1 :
            printf("Enter number of nodes for list1 : ");
            scanf("%d", &n);
            head1 = createlist(n);
            break;
        case 2:
            printf("Enter number of nodes for list 2 : ");
            scanf("%d", &n);
            head2 = createlist(n);
            break;
        case 3 :
            sortlinkedlist(head1);
            displaylist(head1);
            break;
```

```
case 4 :
        head1 = reverseLinkedList(head1);
        displaylist(head1);
        break;

case 5 :
        ~~head~~
        sortlinkedlist(head2);
        displaylist(head2);
        break;

case 6 :
        head2 = reverselinkedlist(head2);
        displaylist(head2);
        break;

case 7 :
        head1 = concatenate linkedlist(head1, head2);
        printf("After concatenation:\n");
        displaylist(head1);
        break;

case 8 :
        Printf("Exiting program...\n");
        exit(0);
default :
        printf("Invalid choice. Try again.\n");
    }
}
return 0;
}
```

O/P

```
1. Create list1
2. Create list 2
3. sort list1
4. reverse list 1
5. Sort list 2
6. Reverse list 2
7. Concatenate
8. Exit
Enter choice: 1
Enter number of nodes for list1: 3
```

data for
Enter node 1 : 10
Enter data for node 2 : 20
Enter data for node 3 : 30
Linked list created successfully.
1. create list 1
2. create list 2
3. sort list 1
4. reverse list 1
5. sort list 2
6. revere list 2
7. concatenate
8. Exit
Enter choice : 2
Enter number of nodes for list 2 : 3
Enter data for node 1 : 68
Enter data for node 2 : 45
Enter data for node 3 : 200
Linked list created successfully
1. create list 1
2. create list 2
3. sort list 1
4. reverse list 1
5. sort list 2
6. reverse list 2
7. concatenate
8. Exit
Enter your choice : 3
Linked list sorted successfully
Linked list : 10 → 20 → 30 → NULL
1. create list 1
2. create list 2
3. sort list 1
4. reverse list 1
5. sort list 2
6. reverse list 2
7. concatenate
8. Exit
Enter your choice : 4
Linked list reversed successfully
Linked list : 30 → 20 → 10 → NULL
1. create list 1
2. create list 2
3. sort list 1
4. reverse list 1
5. sort list 2
6. reverse list 2
7. concatenate
8. Exit
Enter your choice : 5
Linked list sorted successfully
Linked list : 45 → 68 → 200 → NULL
1. create list 1
2. create list 2
3. sort list 1
4. reverse list 1
5. sort list 2
6. reverse list 2
7. concatenate
8. Exit
Enter your choice : 6
Linked list reversed successfully
200 → 68 → 45 → NULL
1. create list 1
2. create list 2
3. sort list 1
4. reverse list 1
5. sort list 2
6. reverse list 2
7. concatenate
8. Exit
Enter your choice : 7
Linked lists concatenated successfully
After concatenation:
Linked list: 30 → 20 → 10 → 200 → 68 → 45 → NULL
1. create list 1
2. create list 2
3. sort list 1
4. reverse list 1
5. sort list 2
6. reverse list 2
7. concatenate
8. Exit
Enter your choice : 8
Exiting program...